Ответы на контрольные вопросы ПР6 (№ 12–13)

Индивидуальный проект «Сервис планирования киносъёмок» (веб-интерфейс «КиноПлан»)

Лимонов Никита Андреевич, ЭФБО-08-24

12. В каких случаях целесообразно выносить часть HTML в отдельный шаблон?

Часть HTML выносят в отдельный шаблон и подключают тегом `{% include %}`, когда это избавляет от дублирования или упрощает чтение основного шаблона. В проекте я вынес фрагменты в четырёх случаях.

1. **Фрагмент повторяется на нескольких страницах.** Навигация нужна на каждой странице, поэтому она лежит в `templates/includes/navigation.html` и подключается один раз в `base.html`. Бейдж статуса съёмки (`shooting_status.html`) выводится на пяти страницах. Если поменять цвет или текст статуса, правка делается в одном файле, а не в пяти.
2. **Фрагмент отображает один объект внутри цикла.** Шаблон списка перебирает коллекцию, а фрагмент описывает один элемент. Например, `film_list.html` выполняет `{% for film in films %}` и подключает `films/includes/film_card.html`. Внутри фрагмента доступна переменная цикла `film`, потому что `include` получает текущий контекст.
3. **У фрагмента есть своя логика отображения.** В `shooting_status.html` стоит `{% if %}/{% elif %}/{% else %}` на три состояния: отменена, уже прошла, запланирована. Страницам, которые подключают бейдж, не нужно знать, как выбирается цвет.
4. **Фрагмент пригодится на новой странице.** Строка таблицы локаций `location_row.html` сделана для списка локаций, а в ПР7 без изменений пригодилась на странице поиска свободных локаций.

Включаемый шаблон	Где используется
`includes/navigation.html`	`base.html` (все страницы)
`shootings/includes/shooting_status.html`	график съёмок, карточка съёмки и (через `shooting_item.html`) главная, страница фильма, страница локации
`shootings/includes/shooting_item.html`	главная, страница фильма, страница локации
`locations/includes/location_row.html`	список локаций, поиск свободных локаций
`films/includes/film_card.html`	список фильмов

Когда выносить не нужно: если фрагмент короткий и встречается в одном месте, отдельный файл только заставит переключаться между файлами. Общий каркас страницы (`<head>`, подключение CSS и JS, подвал) тоже выносится не через `include`, а через

наследование: `base.html` с блоками и `{% extends "base.html" %}` в дочерних шаблонах. `include` вставляет кусок внутрь страницы, а `extends` задаёт рамку, в которую страница вставляет своё содержимое.

13. Для чего используется параметр `name` в `path()`?

Параметр `name` даёт маршруту имя. По этому имени Django сам строит адрес страницы: в шаблоне тегом `{% url %}`, в Python-коде функциями `reverse()` и `redirect()`. Адрес не нужно писать вручную.

```
# locations/urls.py
app_name = "locations"

urlpatterns = [
    path("", views.locations, name="list"),
    path("free/", views.free_locations, name="free"),
    path("<int:location_id>/", views.location_detail, name="detail"),
]
```

```
<!-- шаблон -->
<a href="{% url 'locations:detail' location.id %}">{{ location.name }}
</a>
<!-- результат: href="/locations/1/" -->
```

Зачем это нужно:

- **Адрес задаётся в одном месте.** Если заменить `"free/"` на `"available/"`, все ссылки `{% url 'locations:free' %}` на главной и в списке локаций изменятся сами. С жёстко прописанным `href="/locations/free/"` пришлось бы искать и править каждый шаблон.
- **Параметры подставляются в адрес.** `{% url 'shootings:detail' shooting.id %}` передаёт id в часть маршрута `<int:shooting_id>`.
- **Вместе с `app_name` имя образует пространство имён.** В проекте у приложений `films`, `locations`, `shootings` одинаковые имена маршрутов `list` и `detail`. Они не конфликтуют, потому что полные имена разные: `films:detail`, `locations:detail`, `shootings:detail`.
- **Ошибка видна сразу.** Если опечататься в имени (`'locations:detial'`), Django при отрисовке страницы выдаст `NoReverseMatch`. Опечатка в ручном адресе (`/locaitons/`) проявится только ответом 404, когда пользователь нажмёт на ссылку.

Без `name` маршрут работает, но сослаться на него через `{% url %}` нельзя. Правильность имён проверяет тест `test_named_urls_with_namespaces`: `reverse("films:detail", args=[1]) == "/films/1/"`.